
Principles of Parallel Algorithm Design: Decomposition and Mapping

Milind Chabbi

Nikhil Hegde

Department of Computer Science and Engineering

IIT Dharwad

Limits on Parallel Performance

- What bounds parallel execution time?
 - minimum task granularity
 - e.g. dense matrix-vector multiplication $\leq O(n^2)$ concurrent tasks
 - dependencies between tasks
 - parallelization overheads
 - e.g., cost of communication between tasks
 - fraction of application work that can't be parallelized
 - Amdahl's law
- Measures of parallel performance
 - speedup = T_1/T_p
 - parallel efficiency = $T_1/(pT_p)$

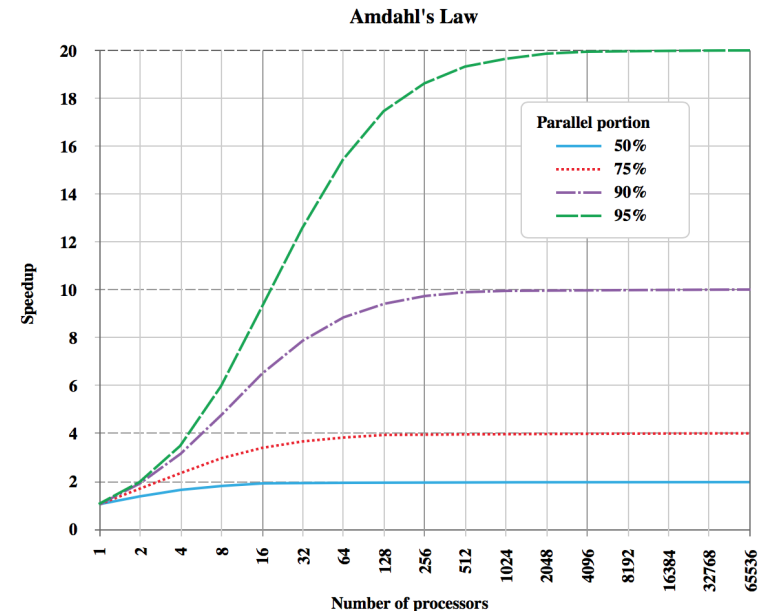
Amdahl's Law

- A hard limit on the speedup that can be obtained using multiple CPUs
- Two expressions of Amdahl's law
 - execution time on N CPUs

$$t_N = (f_p/N + f_s)t_1$$

- speedup on N CPUs

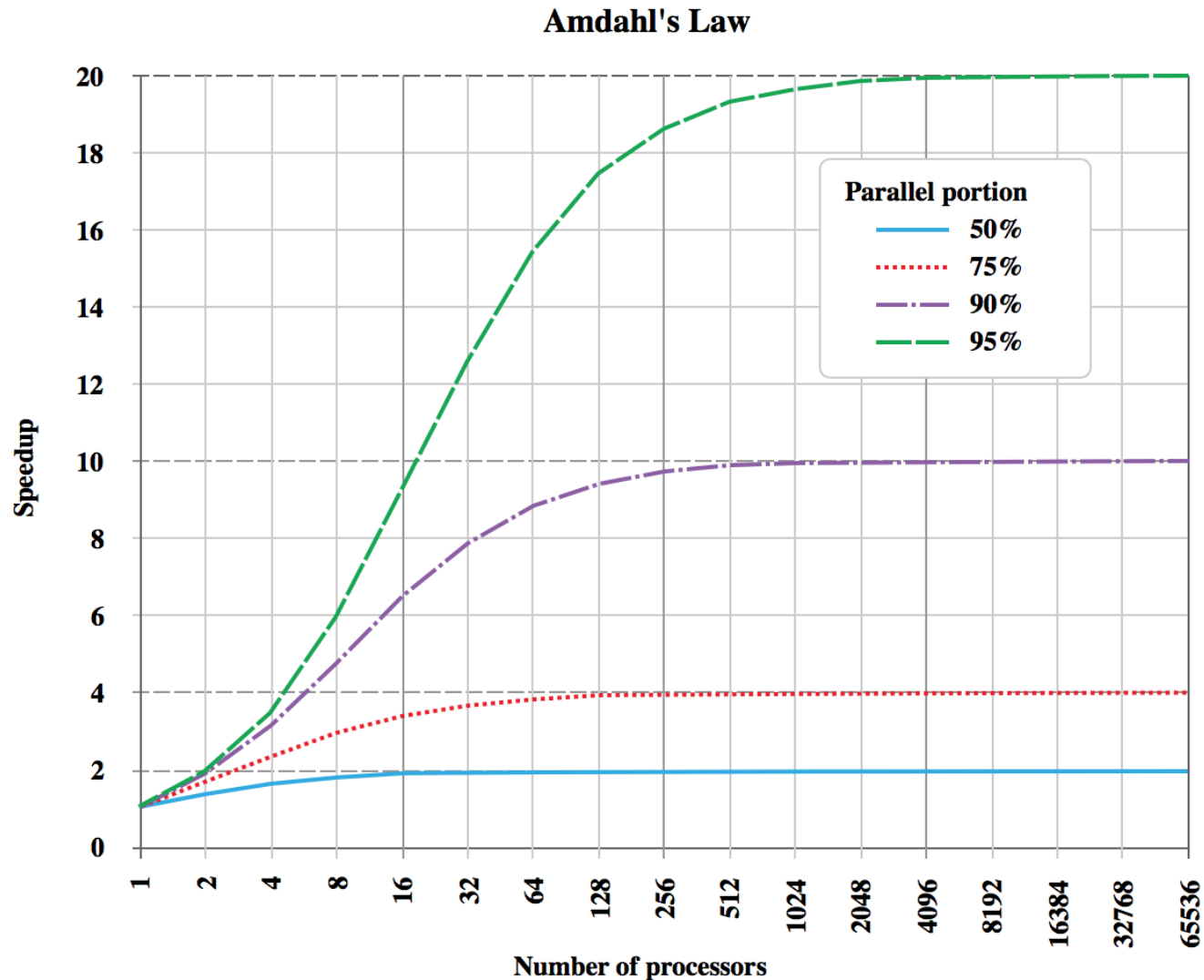
$$S = 1/(f_s + f_p/N)$$



https://en.wikipedia.org/wiki/Amdahl's_law

t_1 : time on 1 CPU
 t_N : time on N CPUs
 f_p : parallel fraction
 f_s : serial fraction = $1 - f_p$

Amdahl's Law



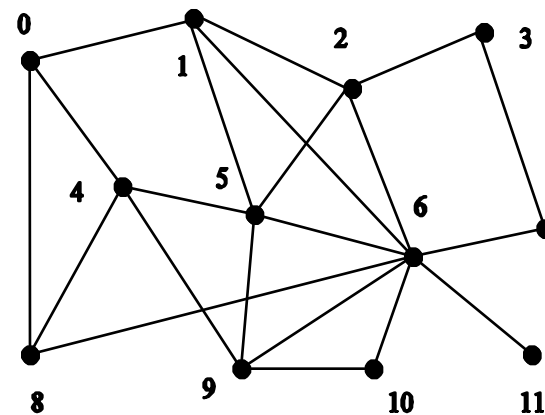
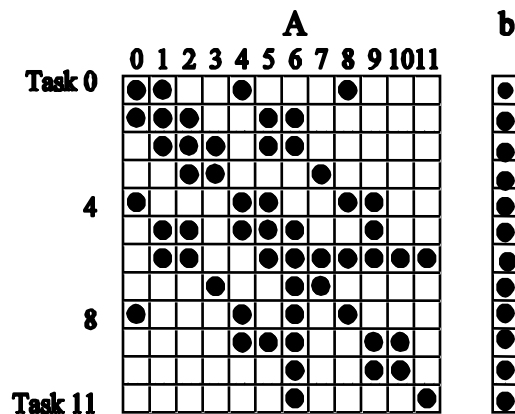
Task Interaction Graphs

- Tasks generally exchange data with others
 - example: dense matrix-vector multiply
 - if vector **b** is not replicated in all tasks, tasks will have to communicate elements of **b**
- Task interaction graph
 - node = task
 - edge = interaction or data exchange
- Task interaction graphs vs. task dependency graphs
 - *task interaction graphs* represent data dependencies
 - *task dependency graphs* represent control dependencies

Task Interaction Graph Example

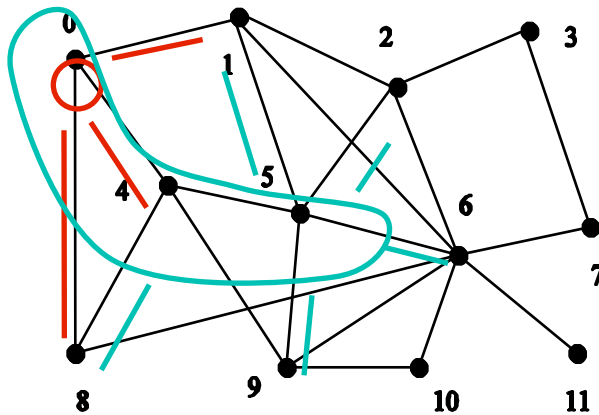
Sparse matrix-vector multiplication

- Computation of each result element = independent task
- Only non-zero elements of sparse matrix A participate
- If, b is partitioned among tasks ...
 - structure of the task interaction graph = graph of the matrix A (i.e. the graph for which A represents the adjacency structure)



Interaction Graphs, Granularity, & Communication

- Finer task granularity increases communication overhead
- Example: sparse matrix-vector product interaction graph



- Assumptions:
 - each node takes unit time to process
 - processing each incident interaction (edge) costs unit time
- If node 0 is a task: communication = 3; computation = 4
- If nodes 0, 4, and 5 are a task: communication = 5; computation = 15
 - coarser-grain decomposition → smaller communication/computation ratio

Tasks, Threads, and Mapping

- **Generally**
 - # of tasks > # threads available
 - parallel algorithm must map tasks to threads
- **Why threads rather than CPU cores?**
 - aggregate tasks into threads
 - thread = processing or computing agent that performs work
 - assign collection of tasks and associated data to a thread
 - operating system maps threads to physical cores
 - operating systems often enable one to bind a thread to a core
 - for multithreaded cores, the OS can bind multiple software threads to distinct hardware threads associated with a core

Tasks, Threads, and Mapping

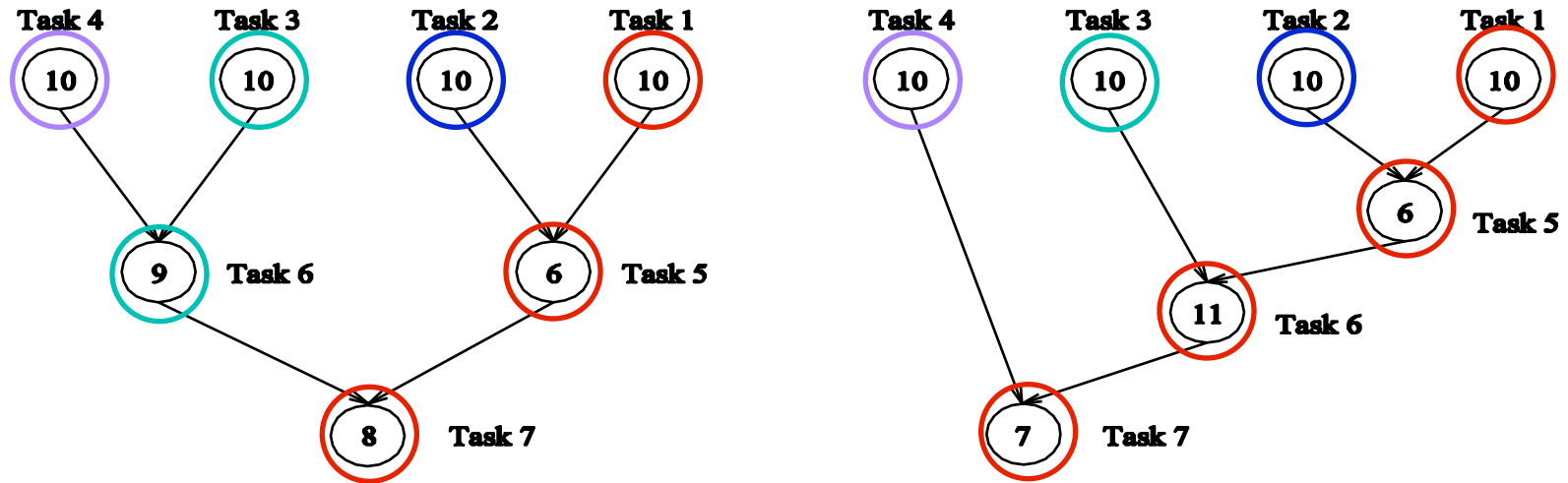
- Mapping tasks to threads is critical for parallel performance
- On what basis should one choose mappings?
 - using task dependency graphs
 - schedule concurrent tasks on separate threads
 - minimum execution time
 - load balance
 - using task interaction graphs
 - want threads to have minimum interaction with one another
 - minimum communication

Tasks, Threads, and Mapping

A good mapping must minimize parallel execution time by

- Mapping independent tasks to different threads
- Assigning tasks on critical path to threads ASAP
- Minimizing interactions between threads
 - map tasks with dense interactions to the same thread
- Difficulty: criteria often conflict with one another
 - e.g. no decomposition minimizes interactions but no speedup!

Tasks, Threads, and Mapping Example



Example: mapping database queries to threads

- Consider the dependency graphs in levels
 - no nodes in a level depend upon one another
 - compute levels using topological sort
- Assign all tasks within a level to different threads

Topics for Today

- **Decomposition techniques**
- **Characteristics of tasks and interactions**
- **Mapping techniques for load balancing**

Decomposition Techniques

How should one decompose a task into various subtasks?

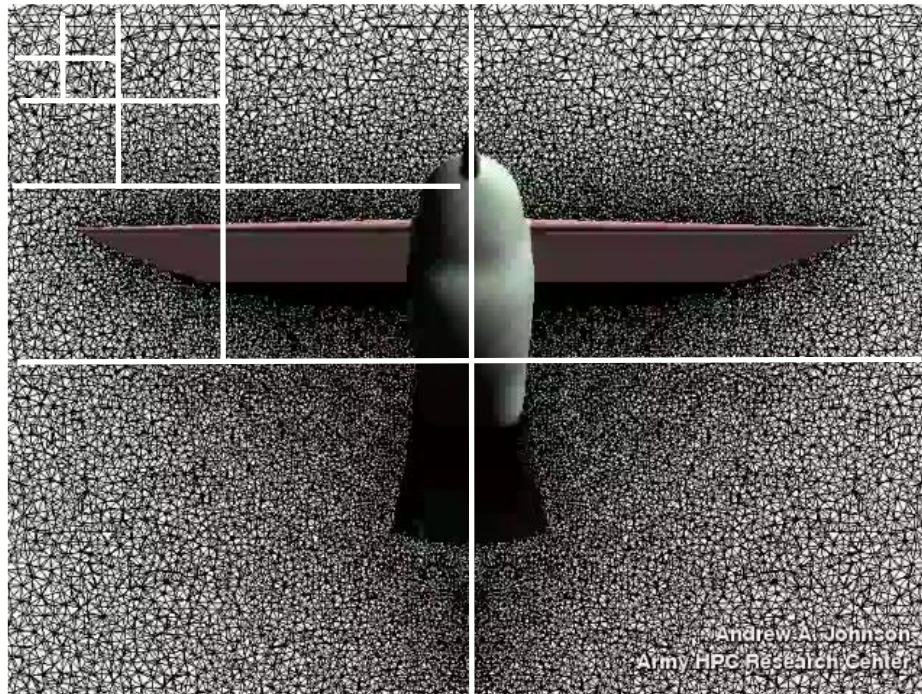
- No single universal recipe
- In practice, a variety of techniques are used including
 - recursive decomposition
 - data decomposition
 - exploratory decomposition
 - speculative decomposition

Recursive Decomposition

Suitable for problems solvable using divide-and-conquer

Steps

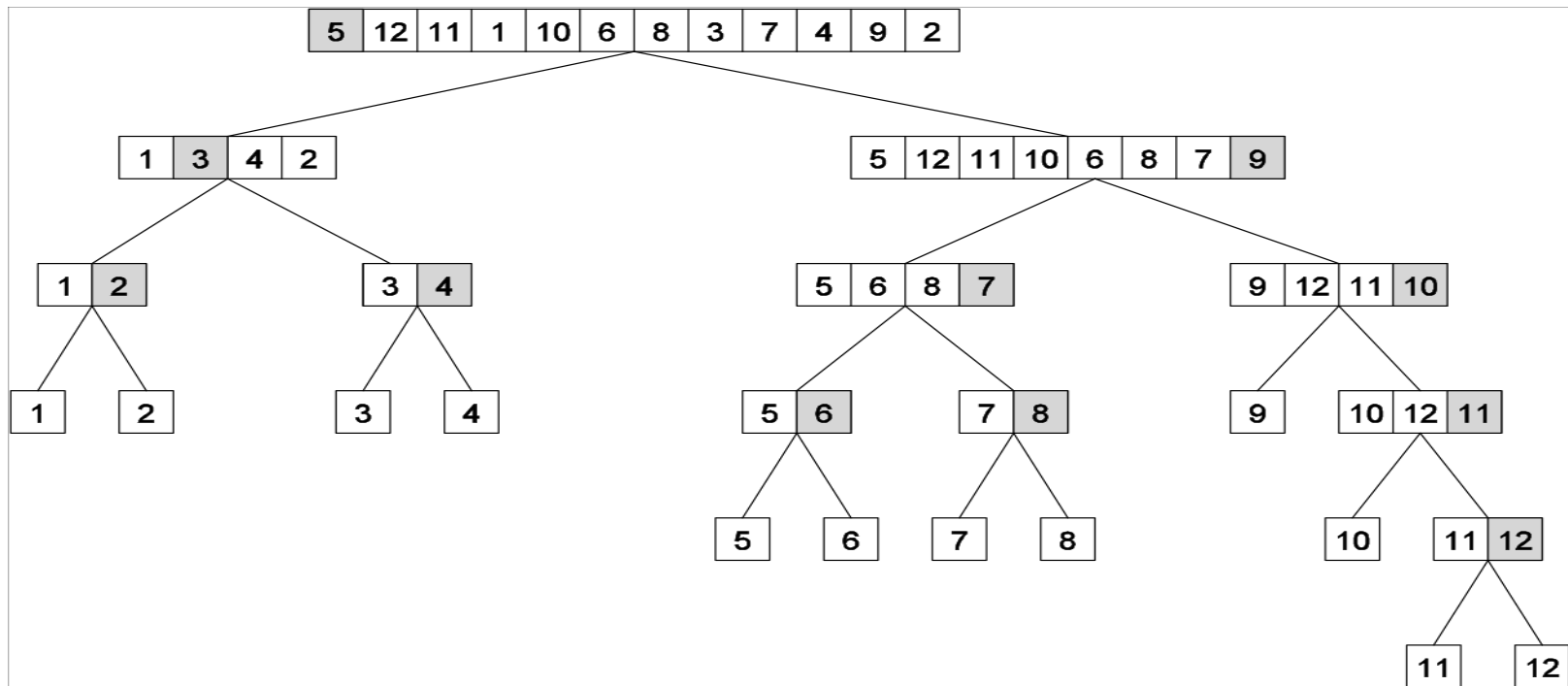
1. decompose a problem into a set of sub-problems
2. recursively decompose each sub-problem
3. stop decomposition when minimum desired granularity reached



Recursive Decomposition for Quicksort

Sort a vector v :

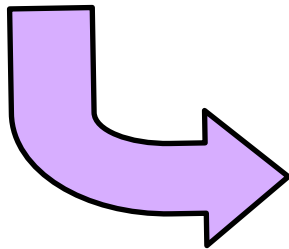
1. Select a pivot
2. Partition v around pivot into v_{left} and v_{right}
3. In parallel, sort v_{left} and sort v_{right}



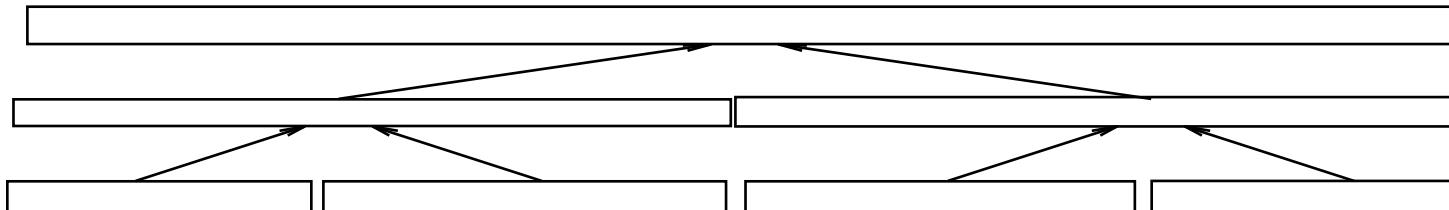
Recursive Decomposition for Min

Finding the minimum in a vector using divide-and-conquer

```
procedure SERIAL_MIN(A, n)  
begin  
  min = A[0];  
  for i := 1 to n - 1 do  
    if (A[i] < min) min := A[i];  
  return min;
```



```
procedure RECURSIVE_MIN (A, n)  
begin  
  if ( n = 1 ) then  
    min := A[0];  
  else  
    lmin := spawn RECURSIVE_MIN(&A[0], n/2 );  
    rmin := spawn RECURSIVE_MIN(&A[n/2], n-n/2);  
    if (lmin < rmin) then  
      min := lmin;  
    else  
      min := rmin;  
  return min;
```



Applicable to other associative operations, e.g. sum, AND ...

Decomposition Techniques

- **Decomposition techniques**
 - recursive decomposition
 - data decomposition
 - exploratory decomposition
 - speculative decomposition
- **Characteristics of tasks and interactions**
- **Mapping techniques for load balancing**
- **Methods for minimizing interaction overheads**
- **Parallel algorithm design templates**

Data Decomposition

- **Steps**
 1. identify the data on which computations are performed
 2. partition the data across various tasks
 - partitioning induces a decomposition of the problem
- **Data can be partitioned in various ways**
 - appropriate partitioning is critical to parallel performance
- **Decomposition based on**
 - input data
 - output data
 - input + output data
 - intermediate data

Decomposition Based on Input Data

- Applicable if each output is computed as a function of the input
- May be the only natural decomposition if output is unknown
 - examples
 - finding the minimum in a set or other reductions
 - sorting a vector
- Associate a task with each input data partition
 - task performs computation on its part of the data
 - subsequent processing combines partial results from earlier tasks

Example: Decomposition Based on Input Data

Count the frequency of item sets in database transactions

Database Transactions	A, B, C, E, G, H	Itemsets	A, B, C	Itemset Frequency
	B, D, E, F, K, L		D, E	
	A, B, F, H, L		C, F, G	
	D, E, F, H		A, E	
	F, G, H, K,		C, D	
	A, E, F, K, L		D, K	
	B, C, D, G, H, L		B, C, F	
	G, H, L		C, D, K	
	D, E, F, K, L			
	F, G, H, L			

- Partition computation by partitioning the set of transactions
 - a task computes a local count for each item set for its transactions

Database Transactions	A, B, C, E, G, H	Itemsets	A, B, C	Itemset Frequency
	B, D, E, F, K, L		D, E	
	A, B, F, H, L		C, F, G	
	D, E, F, H		A, E	
	F, G, H, K,		C, D	
	A, E, F, K, L		D, K	
	B, C, D, G, H, L		B, C, F	
	G, H, L		C, D, K	
	D, E, F, K, L			
	F, G, H, L			

task 1

Database Transactions	A, B, C, E, G, H	Itemsets	A, B, C	Itemset Frequency
	B, D, E, F, K, L		D, E	
	A, B, F, H, L		C, F, G	
	D, E, F, H		A, E	
	F, G, H, K,		C, D	
	A, E, F, K, L		D, K	
	B, C, D, G, H, L		B, C, F	
	G, H, L		C, D, K	
	D, E, F, K, L			
	F, G, H, L			

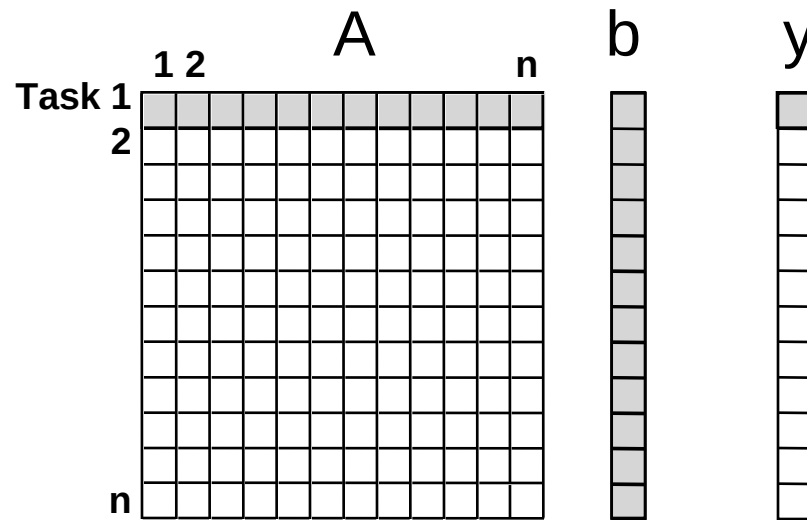
task 2

— sum local count vectors for item sets to produce total count vector

Decomposition Based on Output Data

- If each element of the output can be computed independently
- Partition the output data across tasks
- Have each task perform the computation for its outputs

Example:
dense matrix-vector
multiply



Output Data Decomposition: Example

- **Matrix multiplication: $C = A \times B$**
- **Computation of C can be partitioned into four tasks**

$$\begin{pmatrix} A_{1,1} & A_{1,2} \\ A_{2,1} & A_{2,2} \end{pmatrix} \cdot \begin{pmatrix} B_{1,1} & B_{1,2} \\ B_{2,1} & B_{2,2} \end{pmatrix} \rightarrow \begin{pmatrix} C_{1,1} & C_{1,2} \\ C_{2,1} & C_{2,2} \end{pmatrix}$$

Task 1: $C_{1,1} = A_{1,1}B_{1,1} + A_{1,2}B_{2,1}$

Task 2: $C_{1,2} = A_{1,1}B_{1,2} + A_{1,2}B_{2,2}$

Task 3: $C_{2,1} = A_{2,1}B_{1,1} + A_{2,2}B_{2,1}$

Task 4: $C_{2,2} = A_{2,1}B_{1,2} + A_{2,2}B_{2,2}$

Other task decompositions possible

Example: Decomposition Based on Output Data

Count the frequency of item sets in database transactions

Database Transactions	A, B, C, E, G, H	Itemsets	A, B, C	1	Itemset Frequency
	B, D, E, F, K, L		D, E	3	
	A, B, F, H, L		C, F, G	0	
	D, E, F, H		A, E	2	
	F, G, H, K,		C, D	1	
	A, E, F, K, L		D, K	2	
	B, C, D, G, H, L		B, C, F	0	
	G, H, L		C, D, K	0	
	D, E, F, K, L				
	F, G, H, L				

- Partition computation by partitioning the item sets to count
 - each task computes total count for each of its item sets

Database Transactions	A, B, C, E, G, H	Itemsets	A, B, C	1	Itemset Frequency
	B, D, E, F, K, L		D, E	3	
	A, B, F, H, L		C, F, G	0	
	D, E, F, H		A, E	2	
	F, G, H, K,				
	A, E, F, K, L				
	B, C, D, G, H, L				
	G, H, L				
	D, E, F, K, L				
	F, G, H, L				

task 1

Database Transactions	A, B, C, E, G, H	Itemsets	C, D	1	Itemset Frequency
	B, D, E, F, K, L		D, K	2	
	A, B, F, H, L		B, C, F	0	
	D, E, F, H		C, D, K	0	
	F, G, H, K,				
	A, E, F, K, L				
	B, C, D, G, H, L				
	G, H, L				
	D, E, F, K, L				
	F, G, H, L				

task 2

- append total counts for item subsets to produce result